

Центральная идея доклада Рашера укладывается в простую формулу: мы пишем программы для живых систем мёртвыми методами. Он начинает с автобиографического примера — собственного опыта работы с перфокартами на VAX 11/780 — и методично показывает, как каждый элемент того процесса проецируется на сегодняшний день. Перфокарты стали текстовыми файлами, оператор у считывателя стал CI-сервером, а 45-минутное ожидание результата превратилось в двухчасовой деплой через Docker. Рашер последовательно рассматривает три области, где прогресс отстаёт: управление состоянием и временем (здесь он хвалит Clojure и Erlang), визуальное представление программ (здесь он апеллирует к нейрофизиологии и истории дизайна) и прагматику среды разработки (здесь он противопоставляет интерактивные системы Lisp и Smalltalk традиционному циклу компиляции). Завершает он обзором экспериментальных проектов — Clerk, Hazel, Nest, — которые пытаются сломать инерцию.

Что делает этот доклад по-настоящему сильным — это не отдельные аргументы, а сам метод рассуждения. Рашер берёт вещи, которые кажутся естественными (терминал, текстовый редактор, цикл компиляции), и показывает их историческую случайность. Тильда как символ домашней директории существует не потому, что это хороший дизайн, а потому, что на клавиатуре ADM3A она оказалась рядом с кнопкой Home. Это мощный приём деконструкции. Однако Рашер местами переносит эту логику слишком далеко. То, что практика возникла случайно, не означает, что она плоха. Текстовое представление кода победило не только по инерции — оно прекрасно работает с системами контроля версий, допускает грер, поддаётся автоматической обработке. Рашер сам косвенно признаёт это, когда хвалит Clerk за то, что ноутбуки можно класть в Git.

Доклад натолкнул меня на мысль о скрытой цене интерактивности. Рашер рисует привлекательную картину: вы сидите в живой системе, меняете функцию, видите результат мгновенно. Но у этого подхода есть обратная сторона — воспроизводимость. Когда программа собирается из чистого состояния через детерминированный пайплайн, я могу гарантировать, что коллега получит тот же результат. Когда программа — это накопленное состояние живого образа, в который вносились сотни изменений за сессию, воспроизвести баг становится кошмаром. Smalltalk-сообщество знает это не понаслышке: образы разрастаются, становятся хрупкими, их невозможно осмысленно сравнивать. Мне кажется, что настоящий прорыв случится не тогда, когда мы выберем одну из сторон — живое против мёртвого, — а когда научимся совмещать оба подхода: работать интерактивно, но в любой момент иметь возможность воспроизвести всё с нуля. Именно эту золотую середину, кажется, нащупывает Clerk, и именно поэтому он выглядит наиболее практичным из всех примеров, показанных в докладе.